

ITUNIVERSITY — MODULE M1 · SÉANCE 3

Anatomie d'une application Android

Composants, cycle de vie, Intents, Logcat
Mini-TP « Observer le cycle de vie »



Les quatre composants — les portes d'entrée de votre application

1 Activity

Un écran. L'utilisateur la voit et la touche.

2 Service

Un travail sans écran, qui continue en arrière-plan — synchroniser, jouer de la musique.

3 Broadcast Receiver

Réagir à un événement du système — le réseau revient, la batterie est faible.

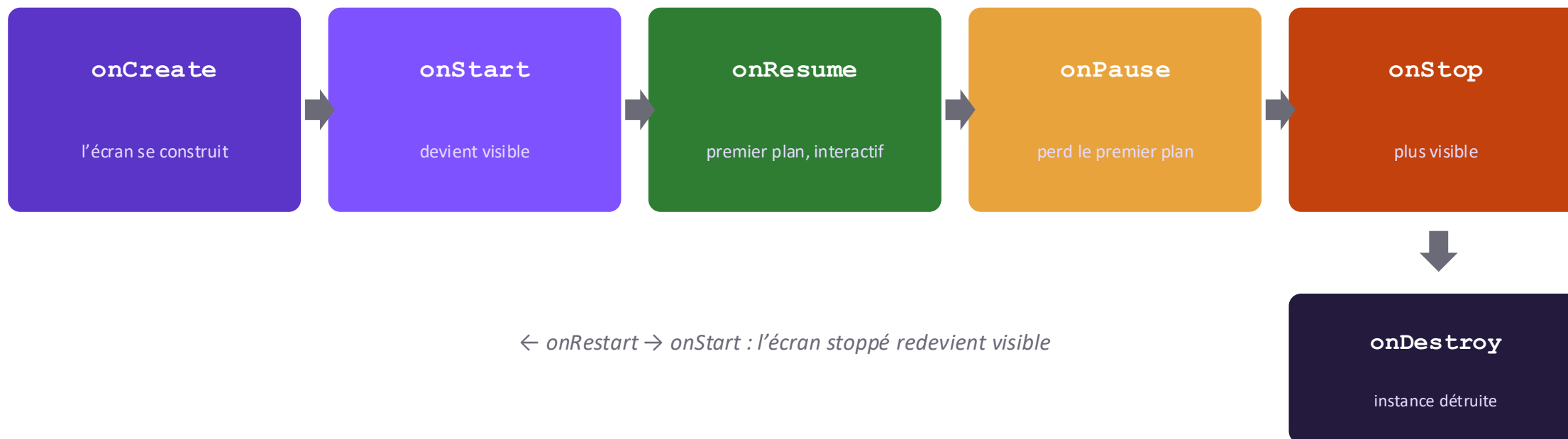
4 Content Provider

Exposer des données aux autres applications — contacts, photos.

Tous déclarés dans le manifest — la carte d'identité de l'application. Aujourd'hui : l'Activity ;.



Le cycle de vie de l'Activity — piloté par le système



Les deux phrases à retenir

Vous n'appellez JAMAIS ces fonctions — le système les appelle, vous les redéfinissez. Et toute ressource prise dans `onResume` se libère dans `onPause`.



Le cas qui surprend tout le monde : tourner l'écran

1 Ce qui se passe vraiment

Le système DÉTRUIT l'Activity et en RECRÉE une neuve : onPause → onStop → onDestroy, puis onCreate → onStart → onResume.

2 La conséquence

Tout ce qui vivait dans l'Activity disparaît — compteur, saisie, liste chargée. Le bug classique : « mon écran se vide quand je tourne le téléphone ».

3 L'ouverture (séance 6)

Il existe un objet qui SURVIT à cette recreation : le ViewModel. Vous allez d'abord VOIR le problème au Logcat — la solution viendra en séance 6.

Aujourd'hui vous verrez le problème de vos yeux — la solution a un nom, et elle arrive en séance 6.



Les Intents : demander un écran — le sien ou celui d'un autre

EXPLICITE — « ouvre CET écran »

```
val intent = Intent(this,
    SecondActivity::class.java)
startActivity(intent)

// Navigation interne :
// on nomme la classe visée.
// (Bouton « Second écran »
// de l'app du mini-TP.)
```

IMPLICITE — « trouve qui sait faire »

```
val intent = Intent(Intent.ACTION_SEND)
    .setType("text/plain")
    .putExtra(Intent.EXTRA_TEXT,
        "Collecte du jour : 4,5 kg")
startActivity(
    Intent.createChooser(intent, null))

// Le système propose les applis
// capables : le sélecteur de partage.
```

La backstack, en une phrase

Chaque écran ouvert s'empile ; le bouton retour dépile. C'est pour cela que « retour » ramène toujours à l'écran précédent.



L'outillage du jour : Logcat, points d'arrêt, stack trace

1 Logcat — le journal de bord

Tout ce que le téléphone raconte. Le réflexe : **FILTRE** — dans le mini-TP, l'étiquette **CYCLE** ne montre que nos callbacks.

2 Points d'arrêt — figer le temps

Un clic dans la marge, le mode débogage, et le programme se fige à la ligne : variables inspectées, exécution pas à pas.

3 Stack trace — lire un crash

Chercher la **PREMIÈRE** ligne qui mentionne notre paquet : c'est presque toujours là. Au-dessus : le mécanisme, pas la cause.

Trois outils installés aujourd'hui, réutilisés à chaque séance du module.



Mini-TP 3 · « Observer le cycle de vie »

1 Prédire

Test « S3 · Prédications » : la séquence exacte des callbacks à la rotation, puis au passage en arrière-plan et retour. Le projet se déverrouille après soumission.

2 Observer au Logcat

Lancer l'app, filtrer sur CYCLE, jouer les deux scénarios, comparer aux prédictions — écarts expliqués par écrit.

3 Compléter le partage

Le bouton « Partager » est vide : y mettre l'Intent implicite vu en cours (ACTION_SEND).

4 Voie ouverte — juger l'IA

Une stack trace fournie ; l'IA propose un diagnostic ; vous jugez : bonne ligne ? bonne cause ? Verdict en 3 lignes au journal.

Vous allez VOIR l'Activity mourir et renaître à la rotation — retenez ce que ça détruit : la séance 6 y répond.

